

- 🚀 零基础保姆级教程：使用 Vercel 免费部署全栈应用 (网页 + FastAPI 后端)
 - 📁 第一步：整理你的目录结构
 - 💻 第二步：编写前端网页 (放在 public 目录下)
 - ⚙️ 第三步：编写 Python 后端 (放在 api 目录下)
 - 🚀 第四步：一键推送到 GitHub 并部署
 - 🏁 第五步：验收你的全栈战果

🚀 零基础保姆级教程：使用 Vercel 免费部署全栈应用 (网页 + FastAPI 后端)

当你在本地写好了一个精美的网页，以及一个可以返回数据的 Python FastAPI 后端时，你该如何把它们放到互联网上，让所有人（包括大屏幕和评委）都能访问到呢？

这时候就需要用到 **Vercel**。

Vercel 是什么？ 它是一个**完全免费、无需绑定信用卡**的云端部署平台。它的神奇之处在于：它不仅能托管你的静态网页，还能同时运行你的 Python 接口！只要你把代码按照规定放在特定的文件夹里，推送到 GitHub 后，Vercel 就会全自动帮你把它变成一个真实的公网网址。

下面我们就一步步教你如何把你的全栈项目搬上 Vercel。

📁 第一步：整理你的目录结构

Vercel 极其智能，你完全不需要去买服务器和配置 Linux。只需遵守它的文件夹命名规则即可：

1. **所有的前端文件**：放在一个名为 **public** 的文件夹里。Vercel 会自动把它们当成静态网页处理。
2. **所有的后端接口代码**：必须放在一个名为 **api** 的文件夹里。只要放在这里，Vercel 就会把它们当成云端接口来运行！

在你的电脑上，新建一个项目文件夹，最终的结构必须长这样：

```
你的项目文件夹/  
├── public/
```

<-- 存放前端网页

├── index.html	<-- 你的主页
├── 你的学号.txt	<-- 黑客松要求的验证文件
├── api/	<-- 存放后端接口
│ ├── index.py	<-- 你的 FastAPI 后端代码
├── requirements.txt	<-- 告诉平台你需要安装哪些 Python 库
└── vercel.json	<-- 告诉平台的路由规则

第二步：编写前端网页 (放在 public 目录下)

在 **public** 文件夹中新建一个简单的 **index.html**。在这个网页里，我们直接去请求咱们自己的后端接口 **/api/hello**：

```
<!-- 文件路径: public/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>我的全栈黑客松项目</title>
</head>
<body>
  <h1>前后端连通测试</h1>
  <button onclick="testApi()">点击请求后端数据</button>
  <p id="result" style="color: blue; font-weight: bold;"></p>

  <script>
    async function testApi() {
      // 直接请求同一个域名下的 /api/hello 接口，告别恼人的跨域问题！
      const response = await fetch('/api/hello');
      const data = await response.json();
      document.getElementById('result').innerText = "后端返回: " +
data.message;
    }
  </script>
</body>
</html>
```

⚠ **参赛必做：**在 **public** 目录下再新建一个 **你的学号.txt**（里面只写你的学号），这是为了能在开发者门户通过自动验证！

第三步：编写 Python 后端 (放在 api 目录下)

在 `api` 文件夹下新建 `index.py` 文件，写下你的 FastAPI 代码。记住：在云端不需要写 `uvicorn.run(...)` 这一行，Vercel 会自动接管你的 `app` 实例！

```
# 文件路径: api/index.py
from fastapi import FastAPI

app = FastAPI()

# 写一个测试接口，注意路径建议带上 /api/ 前缀
@app.get("/api/hello")
def read_root():
    return {"message": "Hello! 这是来自 Vercel 云端 Python 后端的问候!"}

# 极客避坑提示：代码写到这里就结束了！
# 千万不要在底部写 if __name__ == "__main__": uvicorn.run(...)
```

为了让云端知道你需要用到 FastAPI，请在项目根目录新建一个 `requirements.txt` 文件，里面写上：

```
fastapi==0.104.1
pydantic==2.5.2
```

最后，在项目根目录新建一个 `vercel.json` 文件。这个文件是为了告诉 Vercel 所有的接口请求都交给 `index.py` 去处理：

```
{
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/index.py" }
  ]
}
```

第四步：一键推送到 GitHub 并部署

你的前后端代码已经完美合并在一个项目里了，现在把它们搬到云端：

1. **推送代码**：将你整个项目文件夹提交（Commit）并 Push 到你的 GitHub 仓库中。
2. **登录 Vercel**：打开 [Vercel 官网](#)，点击右上角 Sign Up，选择 **Continue with GitHub** 一键登录。
3. **导入仓库**：登录后，点击右侧黑色的 **Add New... -> Project** 按钮。

4. **一键构建**：在列表里找到你刚才上传的黑客松 GitHub 仓库，点击旁边的 **Import** 按钮。
 5. 因为 Vercel 是全自动的，你什么参数都不用改，直接点击蓝色的 **Deploy** 按钮。
 6. 等待约 1 分钟，当屏幕上飘落五颜六色的彩纸时，说明你的**前端和后端都已经自动部署到互联网上了！**
-

第五步：验收你的全栈战果

1. 部署成功后点击 **Continue to Dashboard**，你会得到一个分配给你的公网链接，类似 <https://xxx.vercel.app>。
2. 点击打开它，就能看到你写的网页。
3. 点击网页上的按钮，如果屏幕出现蓝色的字“后端返回：Hello！这是来自 Vercel 云端 Python 后端的问候！”，恭喜你，你的全栈大作已经完美跑通！
4. **大屏提交**：在浏览器里加上 [/你的学号.txt](#)（如 <https://xxx.vercel.app/123456.txt>），如果页面上只显示学号，验证通过！马上把主链接复制到开发者门户点击 **LAUNCH**，生成你的极客海报吧！